



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 25

TITLE : Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Problem Statement

Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle.

Learning Objectives

To create and manage multiple threads using the Thread class.

Theory

A thread is the smallest unit of execution within a Java program. Multithreading enables multiple tasks to execute concurrently, improving application performance and responsiveness. Java provides the Thread class for creating threads by extending it and overriding the `run()` method.

A thread passes through different lifecycle stages including New, Runnable, Running, Blocked/Waiting, and Terminated. The scheduler manages thread execution based on system resources. Methods such as `start()`, `sleep()`, `join()`, and `interrupt()` control thread behavior. Multithreading is commonly used in games, web servers, animation, and background processing applications.

Output Screenshots

```
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp25/Main.java
Thread-1: 1
Thread-0: 1
Thread-0: 2
Thread-1: 2
Thread-0: 3
Thread-1: 3
Thread-0: 4
Thread-1: 4
Thread-0: 5
Thread-1: 5
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

CODE

```
1 package Exp25;
2
3 class MyThread extends Thread {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println(getName() + ": " + i);
7
8             try {
9                 Thread.sleep(500);
10            } catch (InterruptedException e) {
11                System.out.println(e);
12            }
13        }
14    }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         MyThread t1 = new MyThread();
20         MyThread t2 = new MyThread();
21
22         t1.start();
23         t2.start();
24     }
25 }
```

EXERCISE 1

```
1 package Exp25.Exercise;
2
3 class NumberThread extends Thread {
4     public void run() {
5         for (int i = 1; i <= 5; i++) {
6             System.out.println(getName() + ": " + i);
7
8             try {
9                 Thread.sleep(500);
10            } catch (InterruptedException e) {
11                System.out.println(e);
12            }
13        }
14    }
15 }
16
17 public class Exp1 {
18     public static void main(String[] args) {
19         NumberThread t1 = new NumberThread();
20         NumberThread t2 = new NumberThread();
21
22         t1.start();
23         t2.start();
24     }
25 }
```

EXERCISE 2

```
1 package Exp25.Exercise;
2
3 class Bank {
4     int balance = 1000;
5
6     void deposit(int amount) {
7         balance += amount;
8         System.out.println("Deposited: " + amount);
9         System.out.println("Balance: " + balance);
10    }
11
12    void withdraw(int amount) {
13        balance -= amount;
14        System.out.println("Withdrawn: " + amount);
15        System.out.println("Balance: " + balance);
16    }
17 }
18
19 class DepositThread extends Thread {
20     Bank bank;
21
22     DepositThread(Bank bank) {
23         this.bank = bank;
24     }
25
26     public void run() {
27         for (int i = 1; i <= 3; i++) {
28             bank.deposit(500);
29
30             try {
31                 Thread.sleep(500);
32             } catch (InterruptedException e) {
33                 System.out.println(e);
34             }
35         }
36     }
37 }
38
39 class WithdrawThread extends Thread {
40     Bank bank;
41
42     WithdrawThread(Bank bank) {
43         this.bank = bank;
```

```

44     }
45
46     public void run() {
47         for (int i = 1; i <= 3; i++) {
48             bank.withdraw(200);
49
50             try {
51                 Thread.sleep(500);
52             } catch (InterruptedException e) {
53                 System.out.println(e);
54             }
55         }
56     }
57 }
58
59 public class Exp2 {
60     public static void main(String[] args) {
61         Bank bank = new Bank();
62
63         DepositThread deposit = new DepositThread(bank);
64         WithdrawThread withdraw = new WithdrawThread(bank);
65
66         deposit.start();
67         withdraw.start();
68     }
69 }

```

Exercise

Q1. Create a program where multiple threads print numbers simultaneously.

```

dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp25/Exercise/Exp1.java
Thread-0: 1
Thread-1: 1
Thread-1: 2
Thread-0: 2
Thread-1: 3
Thread-0: 3
Thread-1: 4
Thread-0: 4
Thread-0: 5
Thread-1: 5
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp25/Exercise/Exp2.java

```

Q2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously.

```
● dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp25/Exercise/Exp2.java
Deposited: 500
Withdrawn: 200
Balance: 1300
Balance: 1300
Withdrawn: 200
Balance: 1100
Deposited: 500
Balance: 1100
Withdrawn: 200
Deposited: 500
Balance: 1400
Balance: 1400
dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

Conclusion

Thus, the experiment successfully demonstrated the creation and management of multiple threads using the `Thread` class in Java. The lifecycle of threads, including New, Runnable, Running, Waiting/Blocked, and Terminated states, was understood. The use of methods such as `start()`, `sleep()`, `join()`, and `interrupt()` was also studied. The experiment provided a basic understanding of multithreading and concurrent execution in Java.